Musterlösung zur Übung 2

Aufbau von Programmiersprachen

Programmierung 1 · WS 2026/27
Bachelor Cyber Security & Privacy · André Kless, M.Sc.

Hinweis: Die Lösungen zeigen einen möglichen Weg. Andere Grammatiken und Ableitungen sind ebenfalls möglich, sofern sie alle Vorgaben der jeweiligen Aufgabe erfüllen.

Notation: ␣ (in einigen Folien ␣) steht für genau ein Leerzeichen, ↵ für einen Zeilenumbruch, ε für das leere Wort. → bezeichnet eine Produktion, ⊢ einen Ableitungsschritt, ⊢* null oder mehr Schritte. BNF und EBNF verwenden `::=`.

01 Alphabet und Sprache unterscheiden | Basic

a) Zugehörigkeit zu Σ* und L:

Zeichenfolge	In Σ*?	In L?	Begründung
ε	ja	nein	Das leere Wort ist keine Binärdarstellung.
0	ja	ja	Die einzelne Null ist ausdrücklich erlaubt.
101	ja	ja	Nur Binärziffern, keine führende Null.
01	ja	nein	Führende Null in einer mehrstelligen Darstellung.
102	nein	nein	Das Zeichen 2 gehört nicht zum Alphabet.

b) $uv = 1001$, $vu = 0110$, $u\varepsilon = 10$, $|uv| = 4$. Konkatenation ist im Allgemeinen nicht vertauschbar; hier gilt $uv \neq vu$.

c) $|\text{GIB␣0␣AUS↵}| = 3 + 1 + 1 + 1 + 3 + 1 = 10$. Leerzeichen und Zeilenumbruch sind jeweils ein Symbol und haben Länge 1. Dagegen gilt $|\varepsilon| = 0$.

02 Syntax und Bedeutung prüfen | Basic

a–b) Jede Variante startet mit `wert = 4`; jede Codezeile endet mit ↵.

Variante	Code	Syntax	Semantik (Wert und Ziel)
A	`SETZE wert AUF wert * 2`	korrekt	8; das Ziel ist erfüllt.
B	`SETZE wert AUF wert + 2`	korrekt	6; das Ziel ist nicht erfüllt.
C	`SETZE wert wert * 2`	inkorrekt	AUF fehlt zwischen Bezeichner und Ausdruck; kein definiertes Ergebnis.
D	`SETZE wert AUF wert *`	inkorrekt	Nach * fehlt der zweite Wert; kein definiertes Ergebnis.

c) Für B, C und D ist folgende Korrektur möglich:

```
SETZE wert AUF wert * 2
```

Die Syntax legt die zulässige Form fest; bei einem Syntaxfehler ist die Programmbedeutung nicht festgelegt. Die Semantik beschreibt Bedeutung und Wirkung syntaktisch gültiger Anweisungen. Variante B ist syntaktisch korrekt, aber semantisch nicht korrekt für das Ziel: Sie berechnet 6 statt 8.

Basic: Ableitungen und Regelformen

03 Mit einer Grammatik arbeiten | Basic

a) Für `0` genügt ein Schritt: `X ⊢ 0` mit `X → 0`. Für `1011` sind fünf Schritte erforderlich:

Schritt	Ergebnis	Verwendete Produktion
1	`1R`	`X → 1R`
2	`10R`	`R → 0R`
3	`101R`	`R → 1R`
4	`1011R`	`R → 1R`
5	`1011`	`R → ε`

`X ⊢ 1R ⊢ 10R ⊢ 101R ⊢ 1011R ⊢ 1011`

b) Die Produktion `X → 0` beendet die Ableitung sofort. Alle anderen erzeugten Wörter beginnen wegen `X → 1R` mit `1`. Daher ist `01` ausgeschlossen. Auch `ε` ist nicht ableitbar: Jede Startproduktion erzeugt bereits mindestens ein Terminal, das später nicht entfernt werden kann.

c) `1R` ist aus `X` ableitbar, enthält aber noch das Nichtterminal `R`. Es gehört deshalb nicht zu L(G). Ableitbare Zwischenformen dürfen Nichtterminale enthalten; Wörter der erzeugten Sprache bestehen ausschließlich aus Terminalen.

04 Typ 2 von anderen Regelformen abgrenzen | Basic

a) G1 und G2 erfüllen die Typ-2-Bedingung: Links steht in jeder Produktion genau ein Nichtterminal. G3 und G4 erfüllen sie wegen `AB → aB` bzw. `AB → A` nicht.

b) Bei Typ 2 darf rechts eine beliebige Folge aus Terminalen und Nichtterminalen stehen. Deshalb ist `S → aSS` erlaubt. Eine Ableitung lautet:

`S ⊢ aSS ⊢ abS ⊢ abb`

Verwendet werden nacheinander `S → aSS`, `S → b` für das linke und `S → b` für das verbleibende S.

c) Die höchstmögliche Typnummer bezieht sich auf die gegebenen Produktionen:

Grammatik	Typ	Begründung
G1	3	Rechtslinear: S → aS sowie S → b.
G2	2	Kontextfrei; S → aSS verletzt beide Typ-3-Formen.
G3	1	Alle Regeln sind nichtverkürzend; AB → aB verhindert Typ 2.
G4	0	AB → A verkürzt die linke Seite von zwei Symbolen auf eines.

Nicht verwendete Nichtterminale in N verändern diese Einordnung nicht.

Combined: Eine Sprache durch eigene Regeln beschreiben

05 Eine Grammatik für Zugriffscodes entwerfen | Combined

a) Eine mögliche Typ-2-Grammatik ist G = (Σ, N, P, S) mit:

```
Σ = {r, w, 0, 1, #}
N = {S, B, R}
Startsymbol: S

P = { S → rB#, S → wB#,
      B → 0R,  B → 1R,
      R → 0R,  R → 1R,  R → ε }
```

`B` erzwingt das erste Bit. `R` ergänzt beliebig viele weitere Bits. Das abschließende `#` wird bereits durch die Startproduktion genau einmal gesetzt. Jede linke Regelseite ist ein einzelnes Nichtterminal.

b) Jeder Ableitungsschritt wendet genau eine Produktion an:

Zielwort	Vollständige Ableitung
`r0#`	`S ⊢ rB# ⊢ r0R# ⊢ r0#`
`w101#`	`S ⊢ wB# ⊢ w1R# ⊢ w10R# ⊢ w101R# ⊢ w101#`

Für `r0#`: `S → rB#`, `B → 0R`, `R → ε`. Für `w101#`: `S → wB#`, `B → 1R`, `R → 0R`, `R → 1R`, `R → ε`.

c) `r#` scheitert am verpflichtenden ersten Bit. Für `w012#` kann keine Produktion die Ziffer `2` erzeugen. `w10##` ist ausgeschlossen, weil genau eine Startproduktion genau ein `#` liefert und keine andere Regel ein weiteres erzeugt.

Die beiden Startproduktionen ermöglichen `r` und `w`. Durch wiederholtes Anwenden von `R → 0R` oder `R → 1R` entstehen beliebig lange Bitfolgen; `R → ε` beendet jede gewählte Folge.

d) Dieselben Produktionen in BNF, mit `<Zugriffscode>` als Startsymbol:

```
<Zugriffscode> ::= r <Bitfolge> # | w <Bitfolge> #
<Bitfolge>     ::= 0 <Rest> | 1 <Rest>
<Rest>         ::= 0 <Rest> | 1 <Rest> | ε
```

Die Abstände in dieser Darstellung dienen nur der Lesbarkeit. Die erzeugten Zugriffscodes enthalten keine Leerzeichen.

Combined: Boolesche Ausdrücke und BNF

06 Boolesche Ausdrücke beschreiben | Combined

a) Mit dem Startsymbol `<Ausdruck>` ergibt sich:

Zeichenfolge	Zulässig?	Begründung
`a`	ja	Ein einzelnes Atom ist ein Ausdruck.
`!a&&b`	ja	Negation eines Atoms, danach eine Und-Verknüpfung.
`a||b&&c`	ja	Eine Oder-Verknüpfung zweier Und-Ausdrücke.
`!(a||b)`	ja	Ein geklammerter Ausdruck kann negiert werden.
`a&&`	nein	Nach && fehlt ein Faktor.
`||b`	nein	Vor || fehlt der erste Und-Ausdruck.
`a!b`	nein	! ist ein vorangestellter Operator; ein binärer Operator fehlt.

b) Eine gleichwertige BNF ersetzt die beiden Wiederholungen durch Rest-Nichtterminale:

```
<Ausdruck> ::= <Und> <OderRest>
<OderRest> ::= "||" <Und> <OderRest> | ε
<Und>      ::= <Faktor> <UndRest>
<UndRest>  ::= "&&" <Faktor> <UndRest> | ε
<Faktor>   ::= "!" <Faktor> | <Atom>
<Atom>     ::= a | b | c | "(" <Ausdruck> ")"
```

Startsymbol ist `<Ausdruck>`. Zeichen in Anführungszeichen werden genauso erzeugt; die Anführungszeichen selbst nicht. Die runden Klammern in der letzten Regel gehören zur Sprache und dienen hier nicht der EBNF-Gruppierung.

c) Vollständige Linksableitung von `!a||b`; die drei ε-Schritte sind gekennzeichnet:

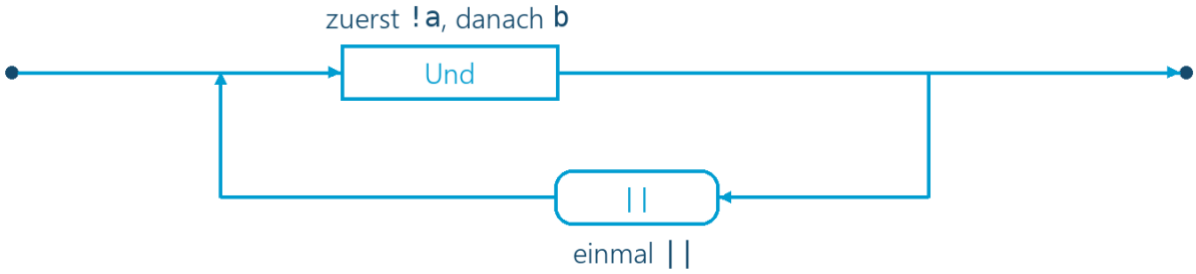
```
<Ausdruck>
⊢ <Und><OderRest>
⊢ <Faktor><UndRest><OderRest>
⊢ !<Faktor><UndRest><OderRest>
⊢ !<Atom><UndRest><OderRest>
⊢ !a<UndRest><OderRest>
⊢ !a<OderRest>                          (ε: <UndRest>)
⊢ !a||<Und><OderRest>
⊢ !a||<Faktor><UndRest><OderRest>
⊢ !a||<Atom><UndRest><OderRest>
⊢ !a||b<UndRest><OderRest>
⊢ !a||b<OderRest>                        (ε: <UndRest>)
⊢ !a||b                                 (ε: <OderRest>)
```

Combined: Syntaxdiagramme

06 Boolesche Ausdrücke — Fortsetzung | Combined

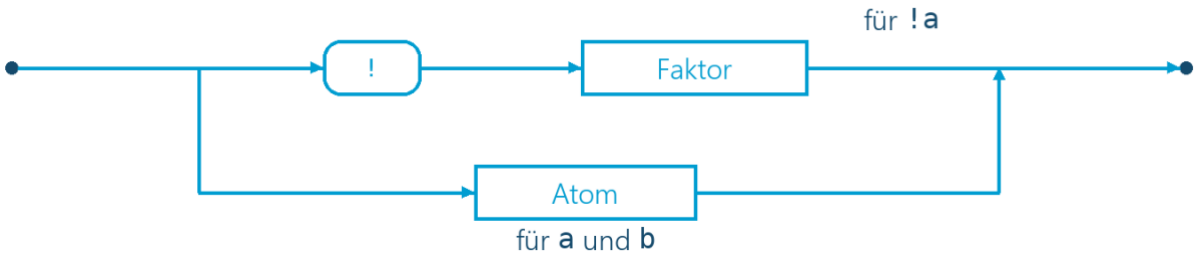
d) Die Diagramme entsprechen der EBNF des Übungsblatts. Abgerundete Kästen bezeichnen Terminale, rechteckige Kästen Nichtterminale. Türkis markiert die für `!a||b` verwendeten Wege.

Syntaxdiagramm für <Ausdruck>



Der Kasten `<Und>` wird zweimal durchlaufen: zuerst für `!a`, danach für `b`. Dazwischen führt der Rückweg einmal durch `||`. Nach dem zweiten `<Und>` geht es zum Ausgang. Jeder `<Und>`-Durchlauf benötigt hier genau einen `<Faktor>`; die Wiederholung über `&&` wird ausgelassen.

Syntaxdiagramm für <Faktor>



`!a` entsteht über den oberen Zweig: Zuerst wird `!` erzeugt. Am Kasten `<Faktor>` beginnt ein weiterer Durchlauf desselben Diagramms am Eingang. Dieser verschachtelte Durchlauf wählt unten `<Atom>` und erzeugt `a`. Danach ist der äußere Durchlauf für `!a` vollständig.

Für das `b` nach `||` wird das `<Faktor>`-Diagramm erneut betreten. Jetzt führt der Weg direkt unten über `<Atom>` zu `b`. Somit wird der obere Zweig einmal und der untere zweimal gewählt.

Combined: Einzelanweisungen in Algo

07 Ausdrücke und Einzelanweisungen beschreiben | Combined

a) Hier gelten wieder die Algo-Regeln. `<Ausdruck>` bezeichnet einen Algo-Ausdruck; die boolesche Sprache aus Aufgabe 6 ist davon unabhängig.

Zeichenfolge	Zulässig?	Begründung
`x`	ja	Ein Bezeichner ist ein Wert und damit ein Ausdruck.
`-7`	ja	Eine erlaubte Ganzzahl ist ein Wert.
`x␣+␣1`	ja	Zwei Werte, ein Operator und beide Leerzeichen.
`x+1`	nein	Die vorgeschriebenen Leerzeichen fehlen.
`x␣+␣1␣+␣2`	nein	Die Regel erlaubt höchstens einen Rechenoperator.
`(x␣+␣1)`	nein	Geklammerte Ausdrücke sind in Algo nicht vorgesehen.
`03`	nein	Führende Null in einer mehrstelligen Ganzzahl.
`-0`	nein	Null ist nur als 0 erlaubt.

b) Die optionale Ergänzung wird durch zwei Alternativen ersetzt:

```
<Ausdruck> ::= <Wert>
             | <Wert> ␣ <Rechenoperator> ␣ <Wert>
```

c) Zuweisung und Ausgabe in BNF:

```
<Zuweisung> ::= SETZE␣<Bezeichner>␣AUF␣<Ausdruck>↵
<Ausgabe>   ::= GIB␣<Wert>␣AUS↵
```

Bei der Ausgabe ist ein `<Wert>` vorgesehen, kein vollständiger `<Ausdruck>`. Der Zeilenumbruch gehört jeweils zur Anweisung.

d) `GIB␣x␣+␣1␣AUS↵` ist deshalb nicht zulässig. Mit der bereits deklarierten Hilfsvariablen `ergebnis` lautet eine gültige Lösung:

```
SETZE ergebnis AUF x + 1
GIB ergebnis AUS
```

Beide Zeilen enden mit ↵. Nur `ergebnis` erhält einen neuen Wert. Die Variable `x` wird gelesen und bleibt unverändert.

Combined: Ein vollständiges Algo-Programm ableiten

08 Ein Algo Programm schrittweise erzeugen | Combined

a) Im Syntaxdiagramm für `<Sequenz>` sind zwei Anweisungen verpflichtend. Für insgesamt vier wird der Rückweg zweimal durchlaufen:

```
<Anweisung> ⊢ <Sequenz>
⊢ <Anweisung><Anweisung><Anweisung><Anweisung>
```

b) Die vier Positionen werden wie folgt entwickelt. Die nicht gezeigten Positionen bleiben im jeweiligen Schritt unverändert:

```
1. <Anweisung> ⊢ <Einzelanweisung> ⊢ <Deklaration>
   ⊢ DEKLARIERE␣<Bezeichner>↲
2. <Anweisung> ⊢ <Einzelanweisung> ⊢ <Eingabe>
   ⊢ LIES␣<Bezeichner>␣EIN↲
3. <Anweisung> ⊢ <Iteration>
   ⊢ SOLANGE␣<Bedingung>:↲␣␣<Anweisung>ENDE␣SOLANGE↲
4. <Anweisung> ⊢ <Einzelanweisung> ⊢ <Ausgabe>
   ⊢ GIB␣<Wert>␣AUS↲
```

Im Rumpf: `<Anweisung> ⊢ <Einzelanweisung> ⊢ <Zuweisung>`. Durch Einsetzen der Zuweisungsregel entsteht folgende Zwischenform; jede angezeigte Zeile endet am expliziten Zeichen ↲:

```
DEKLARIERE␣<Bezeichner>↲
LIES␣<Bezeichner>␣EIN↲
SOLANGE␣<Bedingung>:↲
␣␣SETZE␣<Bezeichner>␣AUF␣<Ausdruck>↲
ENDE␣SOLANGE↲
GIB␣<Wert>␣AUS↲
```

c) Die Bedingung wird vollständig in sieben Schritten entwickelt. Beim Bezeichner wird genau ein Buchstabe gewählt:

```
<Bedingung>
⊢ <Wert>␣<Vergleichsoperator>␣<Wert>
⊢ <Bezeichner>␣<Vergleichsoperator>␣<Wert>
⊢ <Buchstabe>␣<Vergleichsoperator>␣<Wert>
⊢ n␣<Vergleichsoperator>␣<Wert>
⊢ n␣>␣<Wert>
⊢ n␣>␣<Ganzzahl>
⊢ n␣>␣0
```

d) Der Rumpf kann über `<Anweisung> ⊢ <Sequenz>` aus zwei Anweisungen bestehen. Im Syntaxdiagramm für `<Sequenz>` wird der Rückweg dann nullmal durchlaufen. Beide Rumpfzeilen erhalten zwei führende Leerzeichen:

```
DEKLARIERE n
LIES n EIN
SOLANGE n > 0:
  SETZE n AUF n - 1
  GIB n AUS
ENDE SOLANGE
GIB n AUS
```

Challenge: Eine Grammatik für Kfz-Kennzeichen

09 Kfz-Kennzeichen beschreiben | Challenge

a) Eine mögliche Grammatik ist `G = (Σ, N, P, S)`. Als Startsymbol wählen wir `s`. Da die Großbuchstaben A bis Z Terminale sind, verwenden wir Kleinbuchstaben als Nichtterminale. Jedes Nichtterminal besteht aus genau einem Symbol.

```
Σ = {A, ..., Z, 0, ..., 9, -, ␣}
N = {s, o, k, z, b, e, d}
Startsymbol: S = s
```

Produktionen P

```
P = {
  (1) s → o-k␣z,

  (2) o → b,          (3) o → bb,          (4) o → bbb,
  (5) k → b,          (6) k → bb,
  (7) z → e,          (8) z → ed,          (9) z → edd,          (10) z → eddd,

  (11) b → A...Z,
  (12) e → 1...9,
  (13) d → 0...9
}
```

Nur ␣ steht für ein erzeugtes Leerzeichen. Abstände zur Ausrichtung und Regelnummern gehören nicht zum Kennzeichen. Die Kurzschreibweise `b → A...Z` fasst die Produktionen `b → A`, `b → B` usw. bis `b → Z` zusammen. Entsprechend stehen `e → 1...9` und `d → 0...9` jeweils für die Wahl genau einer Ziffer.

Dabei erzeugt `o` das Ortskürzel, `k` die Serie und `z` die Zahl. `b` liefert einen Großbuchstaben, `e` eine erste Ziffer von 1 bis 9 und `d` eine beliebige weitere Ziffer. Die drei Längen werden unabhängig gewählt.

Challenge: Vollständige Ableitungen der Kennzeichen

09 Kfz-Kennzeichen — Fortsetzung | Challenge

b) Jede Zeile zeigt einen einzelnen Ersetzungsschritt. Rechts steht die Regelnummer. Bei den Kurzschreibweisen (11) bis (13) ist zusätzlich die konkret verwendete Produktion angegeben, zum Beispiel `b → B`.

BN-AB␣1234

```
s
⊢ o-k␣z                          (1)
⊢ bb-k␣z                         (3)
⊢ Bb-k␣z                         (11, b → B)
⊢ BN-k␣z                         (11, b → N)
⊢ BN-bb␣z                        (6)
⊢ BN-Ab␣z                        (11, b → A)
⊢ BN-AB␣z                        (11, b → B)
⊢ BN-AB␣eddd                     (10)
⊢ BN-AB␣1ddd                     (12, e → 1)
⊢ BN-AB␣12dd                     (13, d → 2)
⊢ BN-AB␣123d                     (13, d → 3)
⊢ BN-AB␣1234                     (13, d → 4)
```

LEV-A␣1234

```
s
⊢ o-k␣z                          (1)
⊢ bbb-k␣z                        (4)
⊢ Lbb-k␣z                        (11, b → L)
⊢ LEb-k␣z                        (11, b → E)
⊢ LEV-k␣z                        (11, b → V)
⊢ LEV-b␣z                        (5)
⊢ LEV-A␣z                        (11, b → A)
⊢ LEV-A␣eddd                     (10)
⊢ LEV-A␣1ddd                     (12, e → 1)
⊢ LEV-A␣12dd                     (13, d → 2)
⊢ LEV-A␣123d                     (13, d → 3)
⊢ LEV-A␣1234                     (13, d → 4)
```

Beide Ableitungen wählen mit Regel (10) eine vierstellige Zahl. Anschließend werden `e` und die drei Vorkommen von `d` nacheinander ersetzt.

Challenge: Kennzeichen prüfen und Algo erweitern

09 Kfz-Kennzeichen — Begründungen | Challenge

c) Die abgelehnten Wörter verletzen folgende Regeln:

Zeichenfolge	Grund für die Ablehnung
`LEV-AB␣12345`	Die Produktionen (7) bis (10) erzeugen höchstens vier Ziffern; das Wort enthält fünf.
`BN-A␣012`	`e` erzeugt nur Ziffern von 1 bis 9. Eine führende Null kann nicht entstehen.
`BN-␣12`	`k` erzeugt mit (5) oder (6) mindestens einen Großbuchstaben.
`BN-A␣0`	Jede Zahl beginnt mit `e`. Da `e` keine `0` erzeugt, ist die Zahl `0` ausgeschlossen.

10 Algo um eine weitere Schleife erweitern | Challenge

a) Eine passende EBNF-Regel lautet:

```
<Wiederholung> ::= WIEDERHOLE↲␣␣<Anweisung>SOLANGE␣<Bedingung>↲
```

Der Rumpf steht zwei Leerzeichen tiefer als die Schleifenbegrenzungen. Die gezeigten ␣␣ markieren diesen zusätzlichen Einzug: Bei einer mehrzeiligen `<Anweisung>` gilt er für jede Rumpfzeile, zusätzlich zu vorhandenen inneren Einrückungen. Die Rumpfanweisung liefert ihren abschließenden Zeilenumbruch selbst.

b) Die alte Regel bekommt den Namen `<SolangeIteration>`; `<Iteration>` erlaubt beide Formen:

```
<SolangeIteration> ::= SOLANGE␣<Bedingung>:↲
                       ␣␣<Anweisung>ENDE␣SOLANGE↲
<Iteration> ::= <SolangeIteration> | <Wiederholung>
```

Der Umbruch innerhalb der ersten Produktionsdarstellung dient nur dem Satz. Nur die expliziten ↲ werden erzeugt. Dem Algo-Alphabet wird genau der Großbuchstabe **H** hinzugefügt. Die übrigen Buchstaben von `WIEDERHOLE` und `SOLANGE` sind bereits enthalten.

Challenge: Die nachprüfende Schleife anwenden

10 Weitere Schleife — Fortsetzung | Challenge

c) Das Syntaxdiagramm beschreibt die Schreibweise der Schleife:



Die Anweisungen im Schleifenrumpf stehen nur einmal im Programm. Nach jedem Durchlauf wird die Bedingung geprüft: Ist sie wahr, werden dieselben Anweisungen erneut ausgeführt; ist sie falsch, endet die Schleife. Dieses Verhalten gehört zur Bedeutung (Semantik) der Schleife. Weil der Rumpf nur einmal aufgeschrieben wird, braucht das Syntaxdiagramm keinen Rückweg. Die Einrückungsregel aus Teil a) gilt für jede Rumpfzeile.

d) Die Schleife soll weiterlaufen, solange die zuletzt eingelesene ganze Zahl nicht positiv ist. Mit den erlaubten Vergleichsoperatoren lässt sich das als `n < 1` ausdrücken:

```
DEKLARIERE n
WIEDERHOLE
  LIES n EIN
SOLANGE n < 1
GIB n AUS
```

Jede Zeile endet mit einem Zeilenumbruch. Für die Eingabefolge `0, -2, 3` ergibt sich:

Durchlauf	Eingabe n	Bedingung n < 1	Folge
1	0	wahr	Ein weiterer Durchlauf beginnt.
2	-2	wahr	Ein weiterer Durchlauf beginnt.
3	3	falsch	Die Schleife endet; Ausgabe: 3.

Bei sofortiger Eingabe von `3` wird der Rumpf einmal ausgeführt. Danach ist `3 < 1` falsch und das Programm gibt `3` aus.

Die neue Schleife prüft die Bedingung **nach** dem Rumpf; mindestens eine Eingabe ist daher garantiert. Die bisherige `SOLANGE`-Schleife prüft **vor** dem Rumpf und kann ihn nullmal ausführen. Eine entsprechende Lösung mit dieser Schleife benötigt die erste Eingabe vor der ersten Prüfung:

```
DEKLARIERE n
LIES n EIN
SOLANGE n < 1:
  LIES n EIN
ENDE SOLANGE
GIB n AUS
```

Beide Programme liefern für die geprüften Eingabefolgen dasselbe Ergebnis. Eine Bedingung wie `n <= 0` wäre hier nicht zulässig, weil Algo keinen Operator <= vorsieht.